Day 02: Sequence Types — Strings & Tuples

Welcome to Day 2! Today we focus on Python's primary immutable sequence structures: **Strings** (sequences of characters) and **Tuples** (sequences of arbitrary objects). We will explore how they store values, how to access and slice them, and how their immutable nature affects memory management.

Section 1: Strings

1.1 What is a String?

In Python, a **String** is an ordered sequence of Unicode characters representing textual data.

- Strings are **immutable**. Once created, their contents in memory cannot be altered. Any operation that appears to modify a string actually creates a brand-new string object in memory.

1.2 Different Ways to Create Strings in Python

Python provides multiple ways to declare and initialize strings, offering flexibility depending on the content of the text:

A. Single Quotes (`'...'`)

The most basic way to define a string.

```
message = 'Hello, Python!'
```

B. Double Quotes (`"..."`)

Works exactly like single quotes. However, double quotes are useful when your string contains a single quote/apostrophe, as it avoids the need to write escape characters (`\\`).

```
# No escaping needed for the apostrophe
quote = "Python is Guido's creation."

# If single quotes were used, escaping is required:
# quote = 'Python is Guido\\'s creation.'
```

C. Triple Quotes (`'''...'''` or `"""..."""`)

Triple quotes are used for defining **multiline strings** or text containing both single and double quotes.

```
multiline_text = """This is a string
that spans across multiple
```

```
different lines in Python."""
```

*Note: Triple quotes are also used for writing **docstrings** (documentation comments) at the beginning of functions, classes, and modules.*

D. Using the `str()` Constructor (Type Casting)

You can convert other data types (integers, floats, lists, booleans) into their string representations using the built-in `str()` function.

```
age = 25
age_string = str(age)  # Converted to "25"
pi_string = str(3.14)   # Converted to "3.14"
```

1.3 Understanding the `str` Class

In Python, every string we create is an instance (an object) of the built-in class `str`.

```
s = "acts"
print(type(s))  # Output: <class 'str'>
```

Core Characteristics of the `str` Class:

1. Immutability in Memory

When you perform operations on a string object, Python leaves the original string completely untouched in memory. Instead, it computes and registers a new string object elsewhere in memory.

```
original = "Python"
print(id(original))  # E.g., 4381982704

# Modifying the string
modified = original + " 3"
print(id(modified))  # E.g., 4381983584 (a completely new address!)
print(original)      # Still prints "Python"
```

2. The `__str__()` Method

When you invoke the `str(object)` constructor, Python internally looks up and executes that object's `__str__()` special (dunder) method. This method defines how the object should represent itself as a readable text string.

- For example, printing a list object internally uses the list's `__str__()` method to format it inside brackets `[...]`.

3. Inspecting the Class

You can see all methods and attributes exposed by the `str` class in your console using the `dir()` function:

```
print(dir(str))  # Displays all string helper methods
```

And to see full documentation on how to use any method:

```
help(str.split)  # Displays usage info for split()
```

1.4 Accessing Characters in Strings (Indexing)

Since strings are ordered sequences, every character in a string occupies a specific numerical position called an **index**. Python allows you to retrieve individual characters using square brackets `[]` enclosing the index number.

A. Positive Indexing (Zero-Based)

Python uses **zero-based indexing**, meaning the first character of the string starts at index `0`, the second at index `1`, and the last character is at index `len(string) - 1`.

B. Negative Indexing (Backward Counting)

Python also supports **negative indexing** to access elements from right to left.

- The last character of the string is at index `-1`.
- The second-to-last character is at index `-2`.
- The first character is at index `-len(string)`.

C. Visual Representation of String Indexing

Let's look at how the string `"PYTHON"` is indexed:

```
 Character:     P     Y     T     H     O     N
               ---   ---   ---   ---   ---   ---
Positive Idx:   0     1     2     3     4     5
Negative Idx:  -6    -5    -4    -3    -2    -1
```

Code Examples:

```
text = "PYTHON"

# Positive Indexing
print(text[0])  # Output: 'P' (First character)
print(text[2])  # Output: 'T' (Third character)
print(text[5])  # Output: 'N' (Last character)

# Negative Indexing
print(text[-1]) # Output: 'N' (Last character)
print(text[-2]) # Output: 'O' (Second-to-last character)
print(text[-6]) # Output: 'P' (First character)
```

D. Pitfall: The `IndexError`

If you attempt to access an index that is outside the range of the string, Python will raise an `IndexError`.

```
text = "PYTHON"  # len(text) is 6
# print(text[6])  # IndexError: string index out of range
# print(text[-7]) # IndexError: string index out of range
```

Always ensure that your target index is between `-len(string)` and `len(string) - 1`.

1.5 Basic String Operations (Concatenation & Repetition)

Python provides simple operators (`+` and `*`) to combine and multiply text strings.

A. String Concatenation (`+`)

Concatenation means gluing two or more strings together end-to-end. You do this in Python using the plus (`+`) operator.

```
first_name = "Guido"
last_name = "van Rossum"

# Concatenate with a space in between
full_name = first_name + " " + last_name
print(full_name)  # Output: Guido van Rossum
```

Implicit Concatenation

If you place two string **literals** adjacent to each other, Python automatically concatenates them even without the + operator.

```
message = "Hello " "World"
print(message)  # Output: Hello World
```

Note: This only works with literal strings, not with variables.

Pitfall: TypeError on Non-String Concatenation

You cannot concatenate a string with a non-string data type (like an integer or a float) directly. You must cast the non-string to a string first.

```
age = 35
# print("Age: " + age)  # TypeError: can only concatenate str (not "int")
to str

# Fix by casting
print("Age: " + str(age))  # Output: Age: 35
```

B. String Repetition (*)

You can repeat a string a specified number of times using the multiplication (*) operator. The multiplier **must be an integer**.

```
prefix = "la "
chorus = prefix * 3
print(chorus)  # Output: la la la

# Creating a divider line
divider = "-" * 30
print(divider)  # Output: ------------------------------
```

Code Examples:

```
# Combining Concatenation and Repetition
laugh = "Ha"
fun = laugh * 3 + "!"
print(fun)  # Output: HaHaHa!
```

Note: Multiplying a string by 0 or a negative integer returns an empty string "".

1.6 String Formatting

String formatting allows you to insert dynamic variables or expressions into static text strings. In Python, there are three main methods of formatting:

A. C-Style `%` Formatting (Legacy)

The oldest method, borrowing syntax from the C language's `printf` function. It uses format specifiers (like `%s` for string, `%d` for integer, `%f` for float) as placeholders.

```
name = "Rajan"
age = 24
result = "Name: %s, Age: %d" % (name, age)
print(result)  # Output: Name: Rajan, Age: 24
```

Note: This method is legacy and generally discouraged in modern Python because it gets hard to read when handling many variables.

B. The `str.format()` Method (Python 2.6+)

Uses curly braces `{}` as placeholders. You supply variables inside the `.format()` call.

```
name = "Rajan"
age = 24

# Positional formatting
print("Name: {}, Age: {}".format(name, age))

# Positional indexing
print("Age: {1}, Name: {0}".format(name, age))  # Swaps order

# Named keyword placeholders
print("Name: {n}, Age: {a}".format(n="Kishori", a=22))
```

C. F-Strings (Formatted String Literals - Python 3.6+)

The modern, fastest, and most readable string formatting technique. You prefix the string literal with an `f` or `F` and write variable names or expressions directly inside the `{}` braces.

```
name = "Esha"
age = 23
print(f"Name: {name}, Age: {age}")  # Output: Name: Esha, Age: 23
```

D. Advanced F-String Features & "Hacks"

The {} syntax inside f-strings is incredibly powerful and offers several built-in format specifiers and formatting hacks:

1. Self-Documenting Debugging Syntax ({variable=}) (Python 3.8+)

If you append an equal sign = to a variable or expression inside {}, Python prints both the literal expression text and its evaluated value. This is highly useful for debugging and logging.

```
name = "Vinod Kumar"
city = "Bangalore"

# Traditional print debugging
print(f"name={name}, city={city}")  # Output: name=Vinod Kumar,
city=Bangalore

# Using the '=' debugging hack
print(f"{name=}, {city=}")          # Output: name='Vinod Kumar',
city='Bangalore'
```

2. Alignment and Padding (:<, :>, :^)

You can control the width, alignment, and fill character of the text output using format specifiers following a colon ::

- :<width: Left-align within a fixed width (default for strings).
- :>width: Right-align within a fixed width (default for numbers).
- :^width: Center-align within a fixed width.
- Provide a character before the alignment symbol to act as a custom fill character.

```
city = "Bangalore"

print(f"[{city:<15}]")  # Left-aligned:  [Bangalore      ]
print(f"[{city:>15}]")  # Right-aligned: [      Bangalore]
print(f"[{city:^15}]")  # Center-aligned: [   Bangalore   ]

# Using a custom padding fill character (e.g. '*')
print(f"{city:*^17}")   # Center star-padded: ****Bangalore****
```

3. Number Conversions: Binary, Octal, Hex, and Percents

You can convert integers or floats inline into other representations using special formats:

- :b: Binary representation.
- :o: Octal representation.
- :x: Hexadecimal representation.

- `:%`: Percentage representation (multiplies by 100 and formats as %).

```
num = 42
print(f"Binary of {num}: {num:b}")  # Output: 101010
print(f"Hex of {num}: {num:x}")     # Output: 2a

ratio = 0.275
print(f"Percentage: {ratio:.1%}")   # Output: 27.5%
```

4. Inline Datetime Formatting

Instead of importing datetime and calling `.strftime()` to get pretty strings, you can format date/time objects directly inside f-strings:

```
import datetime
today = datetime.date(2026, 8, 26)
print(f"Date: {today:%B %d, %Y}")   # Output: Date: August 26, 2026
```

5. Dictionary Key Lookup and Quote Nesting (Python 3.12+ updates)

How Python handles quotes inside f-string expressions depends on the Python version you are running:

- **Python 3.12 and newer (PEP 701)**: Quote reuse is **fully permitted**. You can use the same quotes inside the `{}` placeholders as the outer string without causing errors.

  ```
  profile = {"name": "Vinod", "city": "Bangalore"}
  # Valid in Python 3.12+
  print(f"City: {profile["city"]}")  # Output: City: Bangalore
  ```

- **Python 3.11 and older**: Reusing the same quotes causes a `SyntaxError` because the interpreter misinterprets the inner quotes as the closing bound of the f-string. You must alternate single and double quotes.

  ```
  # Required for Python 3.11 and older (and good for backward
  compatibility)
  print(f"City: {profile['city']}")  # Output: City: Bangalore
  ```

1.7 Built-in String Methods

The `str` class provides a set of built-in methods to perform manipulations on strings. Here are some of the most commonly used methods, using data related to the name **Vinod Kumar Kayartaya**, email **vinod@vinod.co**, and city **Bangalore**:

---

1. Case Conversions: `.upper()`, `.lower()`, `.title()`

- **`.upper()`**: Converts all characters to uppercase.
- **`.lower()`**: Converts all characters to lowercase.
- **`.title()`**: Capitalizes the first letter of every word.

```
name = "Vinod Kumar Kayartaya"

print(name.upper())  # Output: VINOD KUMAR KAYARTAYA
print(name.lower())  # Output: vinod kumar kayartaya
print("vinod kumar".title()) # Output: Vinod Kumar
```

---

2. Stripping Whitespace: `.strip()`, `.lstrip()`, `.rstrip()`

Removes leading and trailing spaces, tabs, or newlines.

- **`.strip()`**: Removes whitespace from both ends.
- **`.lstrip()`**: Removes from left side only.
- **`.rstrip()`**: Removes from right side only.

```
email = "   vinod@vinod.co   "

print(f"[{email}]")          # Output: [   vinod@vinod.co   ]
print(f"[{email.strip()}]")  # Output: [vinod@vinod.co]
```

---

3. Splitting and Joining: `.split()`, `.join()`

- **`.split(separator)`**: Splits a string into a list of substrings based on the separator (defaults to spaces).
- **`string.join(iterable)`**: Concatenates a list of strings using the primary string as a glue separator.

```
name = "Vinod Kumar Kayartaya"
# Split the string by spaces into a list
name_parts = name.split()
print(name_parts)  # Output: ['Vinod', 'Kumar', 'Kayartaya']

# Join the list parts back using a dash '-'
joined_name = "-".join(name_parts)
print(joined_name)  # Output: Vinod-Kumar-Kayartaya
```

---

4. Search and Index: `.find()`, `.index()`

Used to search for a substring within a string.

- `.find(sub)`: Returns the lowest start index where substring is found. Returns `-1` if not found.
- `.index(sub)`: Same as `.find()`, but raises a `ValueError` if the substring is not found.

```
city = "Bangalore"

print(city.find("galore"))  # Output: 3 (Index of 'g')
print(city.find("Acts"))    # Output: -1 (Not found)
# print(city.index("Acts")) # Raises ValueError: substring not found
```

---

5. Prefix/Suffix Checks: `.startswith()`, `.endswith()`

Returns a Boolean indicating if a string starts or ends with a target pattern.

```
email = "vinod@vinod.co"

print(email.startswith("vinod"))  # Output: True
print(email.endswith(".co"))       # Output: True
print(email.endswith(".com"))      # Output: False
```

---

6. Substring Replacement: `.replace()`

Replaces all occurrences of a target substring with a new substring.

```
city = "Bangalore"

# Replace 'B' with 'M' (Wordplay: Bangalore -> Mangalore)
new_city = city.replace("B", "M")
print(new_city)  # Output: Mangalore
```

Section 2: Tuples

A **Tuple** is a built-in Python sequence type that is **ordered** and **immutable**. Tuples can store multiple items of different data types (heterogeneous data) inside a single variable.

2.1 Defining and Accessing Tuples

1. Defining Tuples

Tuples are written as a list of values separated by commas, usually enclosed in parentheses `()`. Note that in Python, parentheses are technically optional when defining tuples, but they are highly recommended for code readability.

```
# A tuple containing integers
numbers = (1, 2, 3)

# Defined without parentheses (Tuple Packing)
shorthand_tuple = "Vinod", "Bangalore", 560001
print(type(shorthand_tuple))  # Output: <class 'tuple'>

# A tuple containing heterogeneous (mixed) data types
profile = ("Vinod", 25, "Bangalore", True)

# Nested tuples
nested_tuple = ((1, 2), ("a", "b"))
```

Rule: The Single-Item Tuple Comma

If you want to create a tuple that contains only one element, you **must include a trailing comma**. Without the comma, Python treats the parentheses as mathematical parentheses and infers the scalar type of the inner element.

```
not_a_tuple = ("Bangalore")  # Python infers this as a string
print(type(not_a_tuple))     # Output: <class 'str'>

actual_tuple = ("Bangalore",) # Trailing comma marks it as a tuple
print(type(actual_tuple))    # Output: <class 'tuple'>
```

2. Accessing Elements

Like strings, tuples support zero-based positive indexing, negative indexing, and slicing using square brackets `[]`.

```
city_coords = ("Bangalore", 12.97, 77.59)

# Positive indexing
print(city_coords[0])   # Output: 'Bangalore' (First element)

# Negative indexing
print(city_coords[-1])  # Output: 77.59 (Last element)

# Slicing tuples
sub_tuple = city_coords[1:3]
print(sub_tuple)        # Output: (12.97, 77.59)
```

2.2 Operations and Immutability

1. Operations on Tuples

Since tuples are sequences, they support basic concatenation (`+`) and repetition (`*`) operations. Because tuples are immutable, these operations do not modify the original tuples; they return new ones.

```
t1 = (1, 2)
t2 = (3, 4)

# Concatenation
t3 = t1 + t2
print(t3)  # Output: (1, 2, 3, 4)

# Repetition
t4 = t1 * 3
print(t4)  # Output: (1, 2, 1, 2, 1, 2)
```

2. Understanding Immutability

Once a tuple is created in memory, its elements cannot be reassigned, added, or deleted. Attempting to do so raises a `TypeError`.

```
user_info = ("vinod@vinod.co", "Bangalore")

# Attempting to reassign an element
# user_info[1] = "Mangalore"  # TypeError: 'tuple' object does not support
item assignment
```

The Exception: Mutable Objects inside an Immutable Tuple

Immutability applies only to the **references** held by the tuple, not the values inside mutable referents. If a tuple contains a mutable object (like a list), you cannot replace the list object with another object, but you **can** modify the elements inside that list!

```
# A tuple containing an integer and a mutable list
mixed_tuple = (10, [20, 30])

# This is NOT allowed (modifying the tuple reference at index 1)
# mixed_tuple[1] = [40, 50]  # Raises TypeError

# This IS allowed (modifying the contents of the mutable list inside the
tuple)
mixed_tuple[1][0] = 99
print(mixed_tuple)  # Output: (10, [99, 30])
```

2.3 Tuple Packing and Unpacking

Tuple packing and unpacking are powerful features in Python that allow you to bundle values together and separate them into individual variables efficiently.

1. Tuple Packing

When we assign multiple values to a single variable name separated by commas, Python "packs" those values into a single tuple.

```
# Packing values
address = ("vinod@vinod.co", "Bangalore", 560001)
```

2. Tuple Unpacking

Unpacking extracts the values from a tuple and assigns them to individual variables. The number of variables on the left side of the assignment **must match** the number of elements in the tuple.

```
# Unpacking values
email, city, pin = address
print(email)  # Output: vinod@vinod.co
print(city)   # Output: Bangalore
```

Extended Unpacking with the Star (*) Operator

If the number of variables on the left does not match the number of elements in the tuple, you can collect multiple values into a list using the * operator.

```
numbers = (1, 2, 3, 4, 5)

# Collect all middle values into a list
first, *middle, last = numbers
print(first)   # Output: 1
print(middle)  # Output: [2, 3, 4] (List of remaining values)
print(last)    # Output: 5
```

Swapping Variables

Tuple unpacking makes swapping variable values clean and readable without requiring a temporary variable:

```
a = "Vinod"
b = "Bangalore"

# Swap values
a, b = b, a
```

```
print(a)  # Output: Bangalore
print(b)  # Output: Vinod
```